

Research Paper Summaries

Layers, Security Skills, and Technical Expertise

Jaehyuk Lee

Contents

Overview: Skills Across All Papers

The nine papers collectively span four major **system layers** and two broad **security skill domains**:

Layers Touched

- **Hardware / Microarchitecture:** CPU cache internals, DRAM (Rowhammer), dirty-bit logic, PMU counters, cache replacement policy (Tree-PLRU), Intel MEE, SGX EPC.
- **Firmware / TEE Runtime:** ARM TF-RMM, TrustZone Monitor mode, SGX enclave lifecycle instructions (ECREATE, EADD, EENTER, EEXIT), SMMU.
- **Hypervisor / OS Kernel:** KVM, Stage-2 page tables, cgroups, Linux process management, QEMU binary translation, perf/PMU kernel interface.
- **Application / Protocol:** SGX enclave user-space library (`sgxlib`), Solidity smart contracts, DeFi protocol logic, fuzzer runtimes (AFL, libFuzzer).

Security Skills Learned

- **TEE design and exploitation:** Building, attacking, and defending Trusted Execution Environments (SGX, TrustZone, Arm CCA).
- **Microarchitectural side-channel attacks:** Cache timing (Prime+Probe, Prime+Retouch), dirty-bit exploitation, PMU-based leakage.
- **Hardware fault attacks:** Rowhammer against SGX EPC, MEE integrity fault as a denial-of-service primitive.
- **Control-flow attacks:** Return-Oriented Programming (ROP) against TrustZone Secure World.
- **DMA isolation and peripheral security:** Extending CCA's isolation guarantees to SMMU and DMA-capable peripherals.
- **Automated vulnerability discovery:** Coverage-guided fuzzing, dynamic fuzzer composition, runtime scheduling for exploit discovery.
- **Smart contract / DeFi security:** Vulnerability taxonomy, longitudinal exploit analysis, on-chain forensics.

1 PORTAL: Fast and Secure Device Access with Arm CCA

Venue: IEEE S&P 2025

[\[Paper Link\]](#)

Problem

Modern mobile SoCs integrate peripherals (GPU, NPU, DSP) that use DMA to access memory. When confidential VMs run under Arm CCA (Confidential Compute Architecture), these peripherals can break the Realm isolation boundary because the IOMMU/SMMU is not CCA-aware by design.

What Was Built

PORTAL extends Arm CCA's Realm Management Monitor (TF-RMM) to mediate peripheral device access. It introduces a new interface between the RMM and the normal-world hypervisor (KVM) so that Realm VMs can safely share DMA with peripherals without compromising confidentiality or integrity.

Layers Touched

- **Firmware:** TF-RMM — extended to track Realm memory regions and enforce SMMU mappings.
- **Hypervisor:** KVM — new RMI calls to request/release device access on behalf of Realm VMs.
- **Driver:** SMMU driver — modified to enforce Realm-owned PA ranges.
- **Guest VM:** Realm guest driver — userspace API to request DMA from peripherals.

Security Skills

- Arm CCA architecture and Realm Management Interface (RMI)
- Stage-2 page table design for confidential VMs
- SMMU/IOMMU internals and DMA isolation enforcement
- TF-RMM firmware development
- Linux KVM hypervisor modification

2 A Longitudinal Study of Vulnerabilities in the DeFi Ecosystem

Venue: USENIX Security 2024

Problem

DeFi protocols have suffered billions of dollars in losses from smart contract exploits. No prior work systematically studied how vulnerability classes evolve over time and whether the ecosystem learns from past incidents.

What Was Built

A dataset and analysis pipeline covering hundreds of DeFi incidents over multiple years, categorizing vulnerability classes (reentrancy, price oracle manipulation, flash loan attacks, access control flaws, etc.) and tracking recurrence trends. Qualitative coding was performed using Dedoose.

Layers Touched

- **Smart contract layer:** Solidity code auditing and vulnerability classification.
- **Protocol logic:** AMMs, lending protocols, flash loan mechanics.
- **On-chain data:** Blockchain transaction analysis and incident reconstruction.

Security Skills

- Smart contract vulnerability taxonomy and auditing
- Blockchain data analysis and on-chain forensics
- DeFi protocol mechanics (AMMs, flash loans, oracles)
- Qualitative research methodology (Dedoose coding)
- Longitudinal security measurement methodology

3 SENSE: Side-Channel Evaluation and Noise-addition System for Enclaves

Venue: NDSS 2024

Problem

SGX enclaves are vulnerable to microarchitectural side-channel attacks (cache timing, branch predictor leakage, PMU-based observation). Defenses are ad hoc and lack a systematic evaluation framework.

What Was Built

SENSE provides: (1) a systematic framework for evaluating microarchitectural side-channel leakage inside and across SGX enclave boundaries, and (2) a noise-injection mechanism to mitigate observable leakage. It profiles hardware performance counters inside and outside the enclave using the kernel perf subsystem.

Layers Touched

- **CPU microarchitecture:** Cache behavior, branch predictor state, PMU event counting.
- **SGX enclave runtime:** Enclave entry/exit (AEX), SSA-based state observation.
- **OS kernel:** Perf subsystem and PMU access from ring-0.
- **Compiler/toolchain:** Instrumentation of enclave binaries.

Security Skills

- Intel SGX internals (AEX, SSA, EPCM)
- Microarchitectural side-channel attack methodology
- PMU/perf event profiling at kernel level
- GEM5 simulation for microarchitectural analysis
- Side-channel leakage quantification and noise mitigation

4 autofz: Automated Fuzzer Composition at Runtime

Venue: USENIX Security 2023

[\[GitHub\]](#)

Problem

Dozens of fuzzers exist, each optimized for different target characteristics. Manual fuzzer selection is suboptimal, and static combinations waste resources. No prior system dynamically allocates fuzzer compute resources based on live feedback.

What Was Built

autofz monitors coverage feedback (AFL bitmap) from multiple fuzzers running in parallel and applies an AIMD-inspired (Additive Increase, Multiplicative Decrease) scheduler to dynamically reallocate CPU time toward whichever fuzzers are currently finding new coverage.

Layers Touched

- **OS scheduling:** cgroups for CPU quota enforcement, process pinning.
- **Fuzzer internals:** AFL bitmap coverage feedback, libFuzzer corpus sharing.
- **Coverage instrumentation:** Compile-time and binary-level coverage tracking.

Security Skills

- Coverage-guided fuzzing (AFL, libFuzzer, honggfuzz, etc.)
- Linux cgroups and process resource management
- AIMD scheduling algorithm design
- Automated vulnerability discovery methodology
- Runtime profiling and feedback-driven optimization

5 Hacking in Darkness: ROP against TrustZone Secure World

Venue: USENIX Security 2017

Problem

ARM TrustZone's Secure World is assumed to be a strong isolation boundary. This paper demonstrates that ROP (Return-Oriented Programming) attacks can be mounted *against* the Secure World from the Normal World, breaking that assumption.

What Was Built

A full ROP attack chain targeting a real TrustZone implementation. Gadgets were identified in Secure World firmware binaries, chained via SVC/SMC calls from Normal World Linux, and used to achieve arbitrary code execution inside Secure World.

Layers Touched

- **ARM TrustZone:** Monitor mode, Secure EL1 firmware, SVC/SMC dispatch.
- **Normal World OS:** Linux userspace and kernel as the attacker's vantage point.
- **Binary analysis:** Gadget discovery in Secure World binaries.

Security Skills

- Return-Oriented Programming (ROP) exploit construction
- ARM TrustZone architecture (Monitor mode, world switching)
- Binary analysis and gadget finding
- SVC/SMC call interface exploitation
- QEMU-based system emulation for attack development

6 SGX-Bomb: Locking Down the Processor via Rowhammer

Venue: SysTEX 2017

Problem

Intel SGX’s Memory Encryption Engine (MEE) detects EPC bit flips to protect enclave integrity. This defense can be weaponized: an attacker who induces Rowhammer bit flips in EPC causes the MEE to trigger a system-wide processor lockdown, creating a denial-of-service.

What Was Built

SGX-Bomb demonstrates that Rowhammer against EPC pages causes the MEE integrity check to fail, resulting in a machine check exception that locks the processor and requires a physical reboot. This is a novel *liveness attack* using SGX’s own security mechanism against the system.

Layers Touched

- **DRAM hardware:** Rowhammer bit-flip induction via cache eviction hammering.
- **Intel SGX:** EPC memory layout, MEE integrity verification, EPCM state.
- **CPU fault handling:** Machine check exceptions triggered by MEE failure.

Security Skills

- Rowhammer attack methodology (cache flush, hammering patterns)
- Intel SGX internals (EPC, MEE, EPCM)
- Hardware fault injection and exploitation
- Denial-of-service via microarchitectural fault escalation
- Memory subsystem security analysis

7 PrivateZone: Providing a Private Execution Environment using ARM TrustZone

Venue: IEEE TDSC 2016

Problem

Sensitive mobile applications lack hardware-backed isolation equivalent to Intel SGX. ARM TrustZone exists but is locked to OEM/vendor code and unavailable to third-party apps.

What Was Built

PrivateZone opens TrustZone’s Secure World to third-party trusted applications by using Stage-2 paging and Monitor mode to isolate Secure World apps from each other and from the Normal World — providing an SGX-like enclave model on ARM mobile platforms.

Layers Touched

- **ARM TrustZone:** Monitor mode, Secure EL1 OS design, world-switch handler.
- **Stage-2 paging:** Memory isolation between multiple Secure World tenants.
- **Normal World OS:** Linux kernel interface for launching trusted apps.
- **Emulation:** QEMU modifications to support PrivateZone semantics.

Security Skills

- ARM TrustZone design and Secure World OS implementation
- Stage-2 (IPA→PA) page table construction for tenant isolation
- Monitor mode entry/exit and world-switch security
- Trusted application lifecycle management
- QEMU system modification for TEE research

8 OpenSGX: An Open Platform for SGX Research

Venue: NDSS 2015

Problem

Intel SGX was announced before real hardware was available, leaving researchers with no way to build, run, or study SGX enclaves. A faithful, open emulation platform was needed.

What Was Built

OpenSGX is a full software emulation of Intel SGX built on QEMU. It emulates SGX instructions (ECREATE, EADD, EENTER, EEXIT, ERESUME), EPCM, SECS, TCS, and SSA data structures. It also ships `sgxlib`, a user-space library for writing enclave programs, and adds GDB support for in-enclave debugging.

Layers Touched

- **CPU emulation:** QEMU binary translation extended with SGX opcode handling.
- **SGX ISA:** All enclave lifecycle instructions and memory management structures.
- **User-space runtime:** `sgxlib` enclave programming library.
- **Debug tooling:** GDB extension for single-stepping inside emulated enclaves.

Security Skills

- Intel SGX ISA and enclave lifecycle (ECREATE→EENTER→EEXIT)
- QEMU internals and binary translation extension
- Enclave memory management (EPC, EPCM, SECS, TCS, SSA)
- Systems programming for TEE runtime design
- Debugger integration for security research tooling

9 PRIME+RETOUCH: Exploiting Cache Dirty Bits for Stealthy Cache Attacks

Venue: arXiv

Problem

Classic Prime+Probe cache side-channel attacks are noisy because cache eviction is detectable by victim or defense monitors. The write-back cache dirty-bit mechanism is an under-explored side channel that leaks cache-line usage without requiring the noisy eviction step.

What Was Built

PRIME+RETOUCH exploits write-back dirty bits to perform stealthy cache side-channel attacks. By “retouching” (re-dirtying) cache lines, the attacker avoids eviction-based detection used by defenses that monitor cache evictions. The attack is evaluated on Intel and ARM M1 platforms using hardware PMUs and includes a detailed analysis of the Tree-PLRU cache replacement policy.

Layers Touched

- **CPU cache microarchitecture:** L1/LLC write-back dirty-bit logic, cache-line state.
- **Cache replacement policy:** Tree-PLRU analysis to control eviction behavior.
- **PMU:** Hardware performance counters on Intel (TSX) and ARM M1.

Security Skills

- Cache side-channel attack construction (Prime+Probe variant)
- Write-back dirty-bit microarchitectural exploitation
- Tree-PLRU replacement policy reverse engineering
- Intel TSX abuse for cache state observation
- ARM M1 PMU profiling and microarchitectural timing analysis
- Stealth side-channel design (evading eviction-based defenses)

Consolidated Skill Matrix

Skill Area	Papers / Evidence
TEE design & impl.	OpenSGX, PrivateZone, PORTAL
TEE exploitation	Hacking in Darkness (ROP), SGX-Bomb (fault), SENSE (side-channel)
Microarch. side channels	SENSE, PRIME+RETOUCH
Hardware fault attacks	SGX-Bomb (Rowhammer + MEE)
DMA & peripheral isolation	PORTAL (SMMU, CCA)
Hypervisor / Stage-2 paging	PORTAL (KVM), PrivateZone (Stage-2)
Firmware development	PORTAL (TF-RMM), PrivateZone (Monitor mode)
Fuzzing & vuln. discovery	autofz
Smart contract security	DeFi longitudinal study
Emulation / tooling	OpenSGX (QEMU + GDB)
Control-flow exploitation	Hacking in Darkness (ROP gadget chaining)